

Respuestas del Deber 2

César Martínez - Programación Asistida de Aplicaciones

En esta práctica se pasó el divisor de cuenta de Flutter a React. La idea fue mantener lo que hace la aplicación y cambiar la forma de construirla. El trabajo se realizó con ayuda de Codex.

1. ¿Qué porcentaje de la spec se pudo reutilizar?

Se pudo reutilizar el **100%: 64 de 64 enunciados**. No hubo que adaptar ninguno y tampoco se descartó ninguno.

Esto fue posible porque la spec explica lo que debe hacer la aplicación: dividir la cuenta, agregar la propina, validar los datos y mostrar el resultado. No obliga a usar Flutter. Al comparar los archivos, el diff salió vacío, es decir, no encontró cambios.

En analisis_spec.md (analisis_spec.md) está la clasificación completa. Se contaron las reglas por cada aparición, incluyendo las que se repiten en otras secciones; no se contaron títulos ni fechas. Todas quedaron como QUÉ y ninguna como CÓMO o MIXTO. Por eso, el CÓMO fue 0% y no superó el 30% que menciona el deber. Ese 30% es una referencia de la práctica, no una regla general.

El primer build tardó **9 min 50 s** y los seis casos pasaron a los **10 min 9 s**. Estos tiempos sirven como referencia, pero por sí solos no dicen si el trabajo es mejor.

2. ¿Qué pasó con las reglas de la Constitution?

La Constitution es el documento con las reglas para organizar el proyecto. De sus 27 reglas:

Clasificación	Cantidad	Porcentaje
Idénticas	14	51.85%
Adaptadas en redacción	12	44.44%
Reemplazadas	1	3.70%

Se mantuvieron reglas como separar las responsabilidades y no guardar contraseñas en el código. Otras cambiaron de nombres: por ejemplo, `main.dart` pasó a `main.jsx`, y la regla de no usar Flutter en los cálculos pasó a no usar React allí.

La regla que se reemplazó era la de no agregar paquetes externos. En React se necesitan las herramientas que pide el deber, como Vite y Vitest. Se conservaron los cinco principios SOLID, que ayudan a organizar el código y evitar que una parte haga todo.

La comparación **regla por regla, con el motivo de cada cambio**, está en analisis_spec.md (analisis_spec.md), tabla C01-C27. Allí se comparan la Constitution original de Flutter (evidencias/constitution-flutter-original.md) y la Constitution de React (.specify/memory/constitution.md). Se usó la versión original de Flutter, anterior al ajuste para probarlo en Windows.

3. ¿Se modificó algún enunciado de la spec?

No se modificó ninguno. La spec ya separaba lo que debía hacer la aplicación de la tecnología utilizada. La fórmula, las validaciones y los mensajes sirven tanto en Flutter como en React.

Los problemas con las herramientas se resolvieron en la configuración, sin cambiar los requisitos para que las pruebas pasaran. La comparación está en spec-identidad.txt (evidencias/spec-identidad.txt) y la copia inicial se comprueba en spec-inicial-historial.txt (evidencias/spec-inicial-historial.txt). No se volvió a generar la spec.

4. ¿Cambiaron los seis casos de aceptación?

No. Se probaron las mismas entradas y se obtuvieron los mismos resultados en los dos proyectos.

Caso	Datos: monto / personas / propina / modo	Flutter y React
Repartir con propina	100 / 4 / 10 / exacto	27.50
Repartir sin propina	90 / 3 / 0 / exacto	30.00
Cero personas	50 / 0 / 0 / exacto	Debe haber al menos una persona
Monto que no es un número	NaN / 4 / 0 / exacto	Monto inválido
Redondeo exacto	10 / 3 / 0 / exacto	3.33
Redondeo hacia arriba	10 / 3 / 0 / arriba	4.00

NaN representa un dato que no es un número. Cambió la forma de escribir las pruebas por el lenguaje, pero no los casos ni los mensajes. La comparación quedó guardada en `comparacion-casos.json` (`evidencias/comparacion-casos.json`).

5. ¿Qué partes del plan de Flutter cambiaron en React?

Estos son algunos cambios concretos:

1. **La pantalla:** los widgets de Flutter se cambiaron por componentes de React. Se mantuvieron los campos y el botón para calcular.
2. **Los datos de la pantalla:** en Flutter se usaba `setState`; en React se usa `useState` para actualizar los campos y el resultado.
3. **Las pruebas:** se cambió la herramienta de Flutter por Vitest y Testing Library. Los seis casos se mantuvieron.
4. **La organización de archivos:** `lib/` pasó a `src/` y `main.dart` pasó a `main.jsx`. Se siguieron separando la pantalla, los cálculos y las formas de redondear.
5. **La compilación:** en React se usa `npm run build` para preparar la aplicación web.

En total cambiaron **40 de 54 decisiones** del plan y se mantuvieron 14. La comparación del archivo muestra **50 líneas agregadas y 44 eliminadas**. Las líneas y las decisiones se cuentan por separado, porque una decisión puede ocupar varias líneas. El detalle está en `analisis_plan.md` (`analisis_plan.md`).

6. ¿Qué fue lo más y lo menos reutilizable?

Lo más reutilizable fue la **spec**, porque se copió sin cambiar nada. Los seis casos también se mantuvieron en lo que prueban y en sus resultados.

Lo menos reutilizable fue el **código de la aplicación**, porque Flutter usa Dart y React usa JavaScript. Se conservaron la fórmula y la idea del funcionamiento, pero se tuvo que escribir otra vez la pantalla y adaptar los archivos. Entre los documentos, el plan fue de los que más cambiaron: 40 de sus 54 decisiones.

Esto muestra que, en este proyecto, separar lo que hace la aplicación de cómo se construye ayudó a cambiar de tecnología. No permite asegurar que siempre sea más rápido, porque no se registró el tiempo del laboratorio anterior.

Bitácora y cronómetro

Dato	Flutter	React
Tiempo hasta el primer build	No registrado	9 min 50 s (9.83 min)
Tiempo hasta pasar los seis casos	No registrado	10 min 9 s (10.15 min)
Mensajes correctivos después de la petición inicial	No registrado	0
Líneas de código escritas a mano por el estudiante	No registrado	0
Enunciados de la spec modificados	No aplica	0/64
Reglas de la Constitution modificadas	No aplica	13/27
Decisiones del plan modificadas	No aplica	40/54
Líneas del diff del plan	No aplica	+50 / -44
Casos de aceptación aprobados	6/6	6/6
Total de pruebas aprobadas	18/18	36/36

El registro se tomó el **6 de octubre de 2026, hora de Ecuador (UTC-5)**:

- Inicio: **10:26:00**.
- Primer build correcto: **10:35:50**.
- Seis casos aprobados: **10:36:09**.

Se empezó a contar al iniciar la planificación, porque todo se pidió en un solo mensaje. No se inventaron tiempos del laboratorio anterior. El registro completo está en bitacora.md (bitacora.md).

Las pruebas completas de Flutter se hicieron en Chrome por un problema del entorno de Windows. Antes de planificar React se comprobaron los seis casos y las pruebas de redondeo en Dart; las 18 pruebas de Flutter se completaron después, durante el trabajo de React. Ese orden fue distinto al indicado en la guía y queda aclarado aquí.

El código, el análisis y los casos se prepararon con Codex. Por eso aparecen **0 líneas manuales del estudiante**. Los apartados que la guía pide hacer a mano requieren mi revisión personal antes de entregar. Las correcciones de redacción posteriores no forman parte del cronómetro original.

La práctica paso a paso

1. Se revisó el proyecto del laboratorio y sus documentos para entender el divisor de cuenta.
2. Se copiaron los requisitos a React sin cambiarlos y se comprobó que fueran iguales.
3. Se revisaron las reglas del proyecto y se adaptaron las que dependían de Flutter.
4. Se preparó el plan de React y la lista de tareas.
5. Se creó la pantalla y se separaron los cálculos de la parte visual.
6. Se probaron los seis casos, las formas de redondear y el funcionamiento de la pantalla.
7. Se comprobó que la aplicación compilara y se guardaron las mediciones y las evidencias. El trabajo quedó en main, como se pidió.

Para revisar la aplicación, desde la carpeta **Deber 2** se ejecuta `./ejecutar.ps1 dev`. Luego se abre la dirección que muestra la terminal. Por ejemplo, al ingresar monto **100**, personas **4** y propina **10**, el resultado debe ser **27.50** por persona en modo exacto.